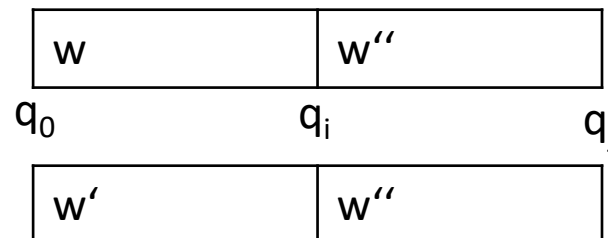


Endliche Automaten II, Nichtdeterministische Maschinen

Pumping-Lemma (endliche Automaten)

- Betrachte die Sprache $L = \{0^n 1^n | n \geq 1\}$ über $\Sigma = \{0,1\}$.
- Kann ein DFA diese Sprache entscheiden?
- Entweder reichen DFAs für diese Sprache mit $2n$ Rechenschritten aus oder kein DFA kann die Sprache L entscheiden.
- Grundlegende Beobachtung: Wenn ein DFA für die Wörter w und w' denselben Zustand erreicht, erreicht er für jedes Wort w'' auch für die Wörter ww'' und $w'w''$ denselben Zustand, das heißt er akzeptiert ww'' und $w'w''$ oder er akzeptiert beide Wörter nicht.



Pumping-Lemma (endliche Automaten)

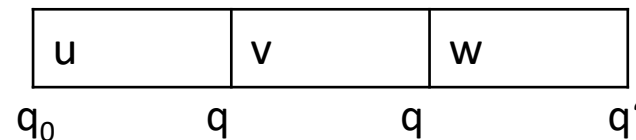
- Nehmen wir an, dass der DFA A die Sprache L entscheidet.
 - Wir betrachten nun die unendlich vielen Wörter $w_n = 0^n$ für $n \geq 1$.
 - Da ein DFA nur endlich viele Zustände haben darf, gibt es zwei Wörter w_i und w_j mit $i \neq j$, so dass A nach dem Lesen von w_i und w_j denselben Zustand erreicht.
 - Für $w'' = 1^i$ ist $w_i w'' \in L$ und $w_j w'' \notin L$.
 - Der DFA A trifft also für eines der Wörter $w_i w''$ oder $w_j w''$ eine falsche Entscheidung.
 - Also gibt es keinen DFA, der L entscheidet.

Pumping-Lemma (endliche Automaten)

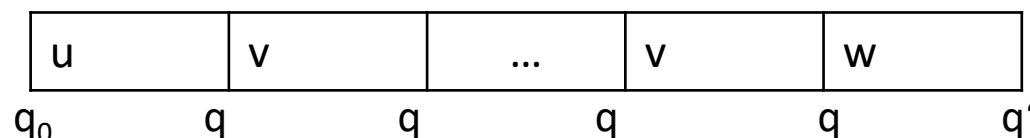
Pumping-Lemma: Sei L eine Sprache, die von einem DFA A akzeptiert wird. Dann gibt es $n \in \mathbb{N}$, so dass sich jedes Wort $z \in L$ mit $|z| \geq n$ in drei Teile zerlegen lässt, das heißt $z = uvw$, wobei $|uv| \leq n$ und $|v| \geq 1$ gilt und $uv^i w \in L$ für alle $i \geq 0$.

Beweis: Wähle $n = |Q|$. Wenn $z \in L$ genügend lang ist, das heißt $|z| \geq n$, durchläuft A auf z genau $|z| + 1 \geq n$ Zustände. Damit wiederholt sich mindestens ein Zustand q beim Lesen der ersten n Buchstaben von z . (Graphentheoretisch wird Kreis in A durchlaufen.)

Sei u der Präfix von z , wenn A zum ersten Mal q erreicht, und uv der Präfix von z , wenn A zum zweiten Mal q erreicht. Dann gilt $|uv| \leq n$ und $|v| \geq 1$. Wähle Suffix w so, dass $z = uvw$. Da $z \in L$ gilt $q' = \delta(q_0, z) \in F$.



Sei nun $uv^i w$ das Eingabewort. Dann gilt $\delta(q_0, uv^i w) = \delta(q, v^i w) = \delta(q, w) = q' \in F$. Also $uv^i w \in L$.



Pumping-Lemma (endliche Automaten)

Beispiel:

$L = \{w \in \{0,1\}^* \mid w = w^R\}$, wobei w^R das Spiegelwort zu w ist, d.h. $w^R = w_k \dots w_1$, falls $w = w_1 \dots w_k$. (Sprache aller **Palindrome**.)

Wir zeigen, dass L nicht regulär ist.

Es sei n die Konstante aus dem Pumping-Lemma. Wir wählen aus L das Wort $z = 0^n 10^n$. Sei dann $z = uvw$ die nach dem Pumping-Lemma existierende Zerlegung mit $|uv| \leq n$, $|v| \geq 1$ und $uv^i w \in L$ für $i \geq 0$. Offensichtlich ist $uv^2 w$ vom Typ $0^{n+j} 10^n$ mit $1 \leq j \leq n$ und damit nicht in L .

Reguläre Ausdrücke

- Reguläre Ausdrücke sind
 - Formeln, die reguläre Sprachen ausdrücken können
 - über Alphabet Σ definiert
 - Basieren auf drei Operationen: Alternative, Verkettung, Wiederholung

- Definition
 1. die leere Menge ist ein regulärer Ausdruck
 2. für alle $a_i \in \Sigma$: a_i ist ein regulärer Ausdruck
 3. sind x und y reguläre Ausdrücke, so sind auch Alternative $(x|y)$, Verkettung (xy) , Wiederholung (x^*) reguläre Ausdrücke

- Ergänzungen
 - man kann auch die positive Wiederholung definieren $x^+ = (x(x^*))$
 - in der Literatur findet man manchmal auch ε als regulären Ausdruck
 - $?$: erkennt vorhergehendes Zeichen oder beliebiges Zeichen aus Σ keinmal oder genau einmal

Reguläre Ausdrücke

- Reguläre Ausdrücke erkennen genau die regulären Sprachen.
- Sie haben einen Nichtdeterminismus, der sich von DFAs nicht so ohne weiteres abbilden lässt.
- Beispiel:
 - seien v, w reguläre Ausdrücke
 - Konkatination vw ist ebenfalls ein regulärer Ausdruck
 - Wann findet der Wechsel von v nach w statt?
- Menschen geben reguläre Sprachen nichtdeterministisch an.
- Idee: bringe Nichtdeterminismus in endliche Automaten.
- Anwendung: z.B. Suche und Ersetze, Syntaxhervorhebung in Editoren

Nichtdeterministische endliche Automaten

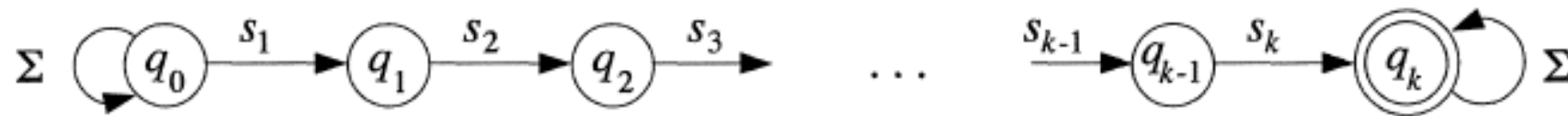
Unterschiede **nichtdeterministischer endlicher Automat NFA** zu DFA:

δ bildet nicht auf **einen** Nachfolgezustand ab, sondern auf **mehrere**.

- Die Überföhrungsfunktion $\delta: Q \times \Sigma \rightarrow Q$ wird durch eine Relation δ auf $(Q \times \Sigma) \times Q$ ersetzt
 - Wir können δ auch als Abbildung von $Q \times \Sigma$ in die Potenzmenge von Q auffassen. Dann ist $\delta(q, a)$ die Menge der Zustände q' , so dass $(q, a, q') \in \delta$ ist.
 - Wie schon für DFAs wollen wir δ auf $Q \times \Sigma^*$ fortsetzen. Es ist $\delta(q, \epsilon) = \{q\}$.
 - Für Wörter w mit $|w| \geq 2$ soll $\delta(q, w)$ die Menge der Zustände sein, die nach dem Lesen von w erreicht werden können. Damit ist für $w = w'a$ mit $a \in \Sigma$ gilt $\delta(q, w) = \bigcup_{q' \in \delta(q, w')} \delta(q', a)$
- Ein NFA A akzeptiert ein Wort w , wenn er nach dem Lesen von w in einem akzeptierenden Zustand **sein kann**, also wenn $\delta(q_0, w) \cap F \neq \emptyset$.

Nichtdeterministische endliche Automaten

- Das einfachste Problem aus der Mustererkennung ist das **String Matching Problem**. Entscheide, ob ein Muster $m = m_1 \dots m_n$ einen String $s = s_1 \dots s_k \in \Sigma^*$, enthält.
- Ein NFA mit nur $k + 1$ Zuständen lässt sich leicht angeben:



- Eine Kante mit Σ bedeutet, dass dieser Übergang für jedes $a \in \Sigma$ möglich ist.
- Der NFA rät wo der String s beginnt, verifiziert dann, dass s tatsächlich in m enthalten ist, und wartet dann das Ende des Wortes ab.

Nichtdeterministische endliche Automaten

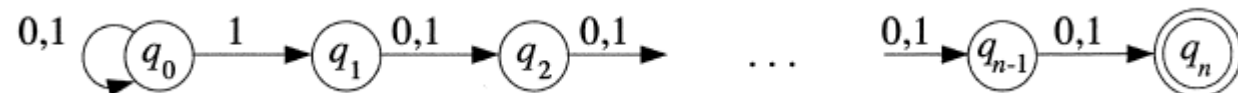
Sind NFAs wirklich DFAs überlegen?

Satz: Sei L_n die Menge aller Wörter über $\{0,1\}$, bei denen der n -letzte Buchstabe eine 1 ist. Dann gilt:

- Es gibt einen NFA für L_n mit $n + 1$ Zuständen.
- Jeder DFA für L_n hat mindestens 2^n Zustände.

Beweis:

Teil 1:



Teil 2: Wegener, 4.4.3.

Nichtdeterministische endliche Automaten

NFAs können nicht mehr Sprachen als DFAs erkennen.

Der letzte Satz ist best case für die Anzahl der benötigten Zustände, um einen NFA durch einen DFA zu simulieren.

Algorithmus (Potenzmengenkonstruktion)

Eingabe: NFA A , beschrieben durch $Q, \Sigma, q_0, \delta, F$.

Ausgabe: DFA A' , der die gleiche Sprache L wie A akzeptiert.

A' hat als Zustandsmenge Q' die Potenzmenge von Q ($|Q'| = 2^{|Q|}$).

Nach dem Lesen von w soll A' in der Menge von Zuständen $\delta(q_0, w)$ sein. Daher setze $\Sigma' = \Sigma$, $q'_0 = \{q_0\}$ und $F' = \{q' \in Q' \mid q' \cap F \neq \emptyset\}$.

Schließlich sei $\delta'(q', a) = \bigcup_{q \in q'} \delta(q, a)$.

Nichtdeterministische endliche Automaten

Um zu zeigen, dass A' ebenfalls die Sprache L akzeptiert, genügt es
 $\delta'(q'_0, w) = \delta(q_0, w)$ für $w \in \Sigma^*$ nachzuweisen.

Beweis mit Induktion über $|w|$:

Für $|w| = 0$ ist $w = \epsilon$ und $\delta'(q'_0, \epsilon) = q'_0 = \{q_0\} = \delta(q_0, \epsilon)$

Für $w = w'a$ gilt nach Induktionsvoraussetzung

$$\delta'(q'_0, w) = \delta'(\delta'(q'_0, w'), a) = \delta'(\delta(q_0, w'), a) = \bigcup_{q \in \delta(q_0, w')} \delta(q, a) = \delta(q_0, w)$$

QED

Nichtdeterministische endliche Automaten

Korrolar: Zu jedem NFA mit n Zuständen gibt es einen äquivalenten DFA mit 2^n Zuständen.

Korrolar: Die Menge der von NFAs erkannten Sprachen sind die regulären Sprachen.

Korrolar: Die Menge der von regulären Ausdrücken erkannten Sprachen sind die regulären Sprachen.

Korrolar: Man kann einen regulären Ausdruck der Länge m in Zeit und Platz $O(2^m)$ kompilieren. Im Anschluss kann man einen String der Länge n in Zeit $O(n)$ nach dem regulären Ausdruck durchsuchen. (Unter der Annahme, dass die oft üblichen Platzhalter für Zahlen bereits ersetzt wurden, was die Länge m erhöhen kann.)

(Es gibt andere Algorithmen für das Suchen mit regulären Ausdrücken, welche die Komplexität in m und n und auch worst case und average case anders abwägen.)

Vorsicht: Laufzeit ist exponentiell in der Eingabelänge, aber nicht exponentiell in der Ausgabelänge. Dieser DFA ist oft nicht minimal, kann aber effizient minimiert werden. Die offensichtlich überflüssigen Zustände kann man vermeiden, siehe Wegener 4.4.6

Es gibt weitere Automatenmodelle, welche die Menge der regulären Sprachen entscheidet.

Eigenschaften und Konstruktionen von Automaten

Satz: Für einen DFA oder NFA kann in linearer Zeit entschieden werden, ob die zugehörige Sprache leer ist.

Beweis:

Die zugehörige Sprache ist genau dann leer, wenn in dem den Automaten darstellenden Graphen vom Anfangszustand q_0 kein Weg zu einem akzeptierenden Zustand existiert.

Teste mit Depth First Suche: Lineare Laufzeit, bezogen auf Größe des Graphen.

QED

Eigenschaften und Konstruktionen von Automaten

Satz: Für einen DFA kann in linearer Zeit entschieden werden, ob die zugehörige Sprache Σ^* ist.

Beweis: Die zugehörige Sprache ist genau dann Σ^* , wenn in dem den Automaten darstellenden Graphen der Anfangszustand q_0 akzeptiert und von ihm kein Weg zu einem nicht akzeptierenden Zustand existiert.

Wieder mit Depth First Suche testen.

QED

Eigenschaften und Konstruktionen von Automaten

Satz: Für einen DFA kann in linearer Zeit entschieden werden, ob die zugehörige Sprache unendlich viele Wörter enthält.

Beweis:

Entferne alle überflüssigen Zustände (lineare Zeit).

Akzeptiere unendlich viele Wörter genau dann, wenn der darstellende Graph einen Kreis enthält und von diesem Kreis aus ein akzeptierender Zustand erreicht werden kann.

Wenn so ein Kreis existiert, kann man diesen von q_0 aus erreichen, beliebig oft durchlaufen und schließlich einen akzeptierenden Zustand erreichen.

Wenn es derartige Kreise nicht gibt, können die akzeptierenden Zustände nur auf kreisfreien Wegen, deren Länge durch $|Q|$ beschränkt ist, erreicht werden. Damit ist die zugehörige Sprache endlich.

Suche nun rückwärts von akzeptierenden Zuständen aus, ob wir einen Kreis finden.

- Details (warum ist das linear?), siehe Wegener, 4.6.3

QED

Eigenschaften und Konstruktionen von Automaten

Satz: Aus einem DFA A für eine Sprache L kann in linearer Zeit $O(|Q|)$ ein DFA für $\bar{L}$, das Komplement von L , erzeugt werden. Insbesondere ist $\bar{L}$ regulär, wenn L regulär ist.

Beweis: Offensichtlich genügt es, die Menge F der akzeptierenden Zustände durch $Q - F$ zu ersetzen.

QED

Falls für L ein NFA A gegeben ist, ist die Konstruktion eines NFA für L nicht einfach möglich. Für Wörter $w \in L$ kann es in A akzeptierende und nicht akzeptierende Wege geben. Im NFA für $\bar{L}$ darf es für w keinen akzeptierenden Weg geben.

Eigenschaften und Konstruktionen von Automaten

Satz: Aus zwei DFAs A_1 und A_2 für die Sprachen L_1 und L_2 kann in Zeit $O(|Q_1||Q_2||\Sigma|)$ ein DFA A für die Vereinigung der Sprachen L_1 und L_2 konstruiert werden. Insbesondere ist $L_1 \cup L_2$ regulär, wenn L_1 und L_2 regulär sind. Gleiches gilt für den Durchschnitt regulärer Sprachen.

Beweis mit Produktautomaten:

Sei $Q = Q_1 \times Q_2$, q_0 das Paar der Anfangszustände von A_1 und A_2 und F die Menge aller (q_1, q_2) mit $q_1 \in F_1$ oder $q_2 \in F_2$. Schließlich sei

$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$$

Für den Durchschnitt können wir eine analoge Konstruktion benutzen oder die de Morgan Regel anwenden.

QED

Eigenschaften und Konstruktionen von Automaten

Satz: Für zwei DFAs A_1 und A_2 für die Sprachen L_1 und L_2 kann in Zeit $O(|Q_1||Q_2||\Sigma|)$ der Gleichheitstest entschieden werden, also entschieden werden, ob $L_1 = L_2$ ist.

Beweis:

$$L_1 = L_2 \Leftrightarrow (L_1 \cap \bar{L}_2 = \emptyset) \wedge (\bar{L}_1 \cap L_2 = \emptyset)$$

Für $L_1 \cap \bar{L}_2$ und $\bar{L}_1 \cap L_2$ können in der vorgegebenen Zeit endliche Automaten konstruiert werden (s.o.).

Für jeden der Automaten kann in linearer Zeit getestet werden, ob er die leere Sprache akzeptiert (s.o.).

Es ist $L_1 = L_2$ genau dann, wenn beide Tests positiv ausgehen.

QED

Analog kann entschieden werden, ob $L_1 \subseteq L_2$ ist. Dies ist äquivalent zu $L_1 \cap \bar{L}_2 = \emptyset$.

Die gleiche Frage für NFAs ist NP-hart.

ϵ -Kanten in NFAs

- Manchmal erlaubt man ϵ -Kanten in NFAs. So können manche Konstruktionen leichter angegeben werden.
- Übung: Die ϵ -Kanten können einfach entfernt werden, so dass man einen äquivalenten NFA erhält.

Eigenschaften und Konstruktionen von Automaten

Es seien L_1 und L_2 Sprachen über Σ . Die **Produktsprache** (auch: **Konkatenation**) ist $L_1L_2 = \{vw \in \Sigma^* \mid v \in L_1, w \in L_2\}$

Satz: Aus zwei DFAs A_1 und A_2 für die Sprachen L_1 und L_2 kann in Zeit $O((|Q_1| + |Q_2|)|\Sigma|)$ ein NFA für L_1L_2 konstruiert werden. Insbesondere ist L_1L_2 regulär.

Beweis:

Der NFA A für L_1L_2 arbeitet auf der Zustandsmenge $Q_1 \cup Q_2$, das Alphabet ist Σ , Anfangszustand ist der Anfangszustand aus Q_1 .

Die Menge der akzeptierenden Zustände ist gleich F_2 , falls $\epsilon \notin L_2$ ist, und gleich $F_1 \cup F_2$, falls $\epsilon \in L_2$.

Die Zustandsüberföhrungsfunktion δ rät, an welcher Stelle der Eingabe w der Präfix aus L_1 gelesen ist und der Suffix aus L_2 beginnt.

- Für $q \in Q_2$ sei $\delta(q, a) = \{\delta_2(q, a)\}$.
- Für $q \in Q_1 - F_1$ sei $\delta(q, a) = \{\delta_1(q, a)\}$
- Für $q \in F_1$ sei $\delta(q, a) = \{\delta_1(q, a), \delta_2(q^*, a)\}$ für den Anfangszustand q^* von A_2

Eigenschaften und Konstruktionen von Automaten

Für eine Sprache L über Σ ist der **Kleenesche Abschluss** $L^* = \bigcup_{i \geq 0} L^i$, wobei L^i das i -fache Produkt von L ist mit $L^0 = \{\epsilon\}$. Der **positive Abschluss** ist $L^+ = \bigcup_{i \geq 1} L^i$.

Satz: Aus einem DFA A für L kann in Zeit $O(|Q||\Sigma|)$ ein NFA für L^* konstruiert werden. Insbesondere ist L^* regulär, wenn L regulär ist.

Beweis: Konstruiere NFA für die Sprache L^+ und füge ϵ hinzu. Die Konstruktion folgt der Idee aus dem Beweis der Konstruktion von $L_1 L_2$.

Wir können häufiger raten, wann wir zur nächsten Iteration von L weitergehen.

Details: Übung.

QED

Analog für L^+ .

Übung: gebe eine leichtere Konstruktion mit Hilfe von ϵ -Kanten an.